

Unity 3D

Putting a GUI onto a game

- به واسطه بصری بازی ها کامپیوتر مانند دکمه تنظیم و متن ها (GUI short for Graphical User Interface) گفته می شود.

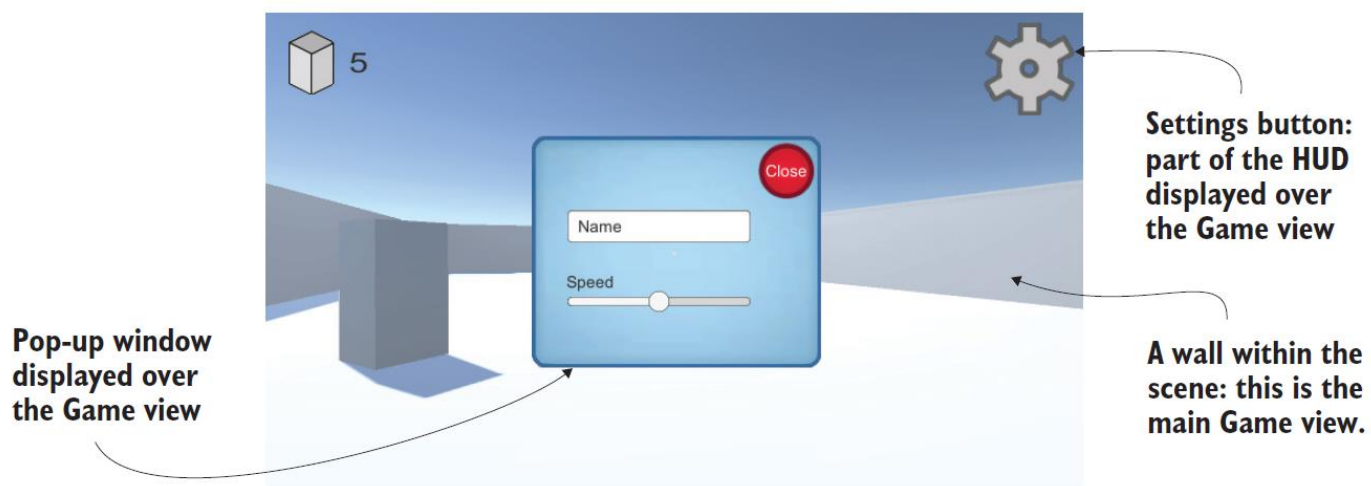


Figure 7.1 The GUI you'll create for a game

این واسطه ها اغلب در بالای صفحه نمایش بازی نشان داده می شوند به همین دلیل به آنها *heads-up display (HUD)* گفته می شود

- در این فصل تلاش کرده است برای بازی ساخته شده فصل سوم یک واسطه نمایش وضعیت در بالا (HUD) بسازد
- اگر به یاد داشته باشید در بازی اول شخص فصل قبل ، موس از دید کاربرد مخفی شده بود که در این شرایط امکان کار کردن با GUI وجود نخواهد داشت
- بنابراین در هنگام کار کردن با GUI ما باید موس را فعال کنیم
- کد زیر یک دکمه را در صفحه نمایش رسم می کند.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
void OnGUI()
{
    if (GUI.Button(new Rect(100, 100, 40, 20), "Test"))
    {
        Debug.Log("Test button");
    }
}
```

تابع OnGUI بعد از آن که رندر تمام اجسام و صحنه سه بعدی انجام شد فراخوانی میشود تا منو GUI را رسم کند.

یک Game object تهی ساخته و کد بالا را به آن اضافه کنید

- امروزه تلاش می شود از قابلیت های ۲ بعدی نسخه های جدید یونیتی برای ساخت منو استفاده شود.
- سیستم رابط کاربری جدیدتر در حالتی کنترل شده تر کار می کند، بنابراین گرافیک ها یک بار چیده می شوند و سپس هر فریم بدون نیاز به تعریف مجدد مداوم ترسیم می شوند. در این سیستم، گرافیک های رابط کاربری در ویرایشگر **Unity** قرار می گیرند. این دو مزیت را نسبت به رابط کاربری حالت قدیمی تر دارد: (۱) هنگام قرار دادن عناصر رابط کاربری می توانید ببینید که رابط کاربری چگونه به نظر می رسد، و (۲) این سیستم سفارشی کردن رابط کاربری با تصاویر خود را آسان می کند.

- می‌خواهیم یک نمایش امتیاز و یک دکمه تنظیمات را در گوشه‌های صفحه روی نمای اصلی بازی قرار دهیم. دکمه تنظیمات یک پنجره پاپ آپ ظاهر می‌شود که دارای یک فیلد متنی و یک نوار لغزنده است.
- در این مثال، از کنترل‌های ورودی برای تنظیم نام و سرعت حرکت بازیکن استفاده می‌شود، اما در نهایت آن عناصر رابط کاربری می‌توانند تنظیمات مربوط به بازی شما را کنترل کنند.

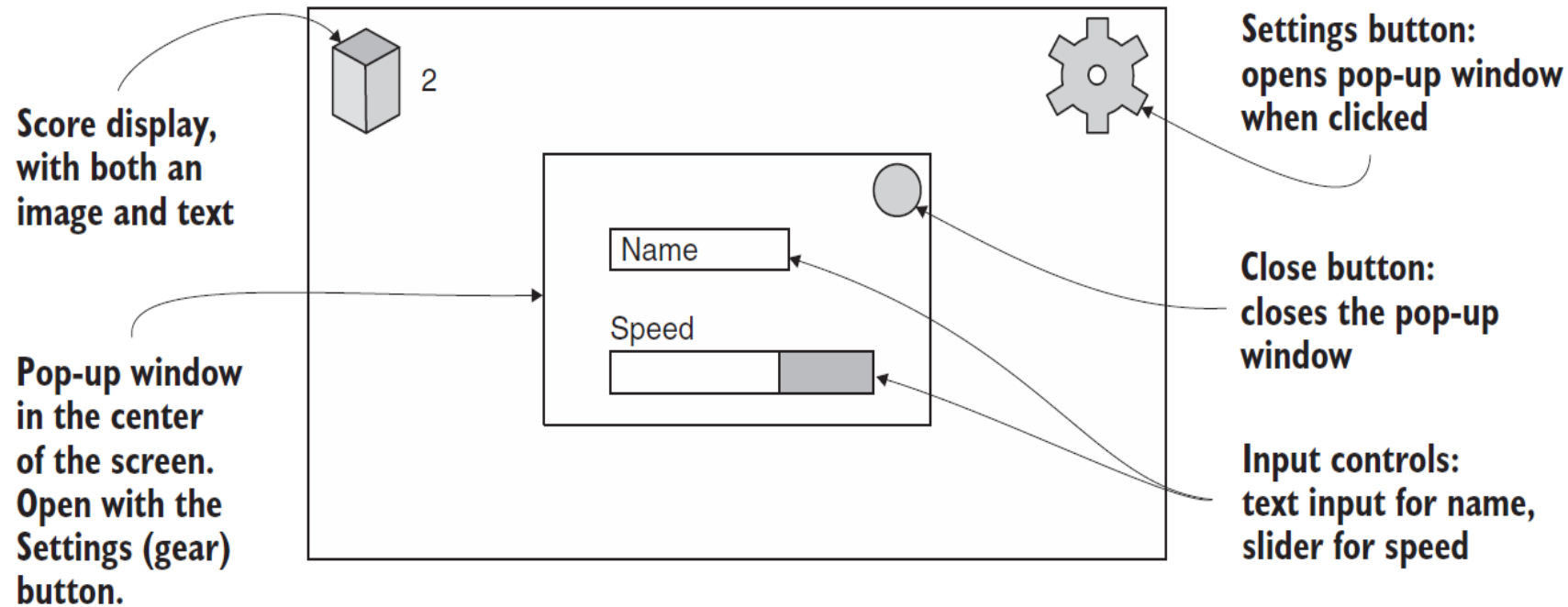


Figure 7.2 Planned GUI

- این رابط کاربری برای نمایش مواردی مانند دکمه‌ها به تعدادی تصویر نیاز دارد. رابط کاربری از تصاویر ۲ بعدی ساخته شده است، بنابراین شما دو مرحله را دنبال خواهید کرد:
 - وارد کردن تصاویر دو بعدی به پروژه (به این تصاویر Sprite گفته می‌شود)
 - قرار دادن Sprite ها در صحنه (کشیدن و رها کردن آنها در صحنه)
- تصاویر مورد نیاز خود را به پروژه وارد کرده و خصوصیت Texture Type آنها را به Sprite (2D and UI) تغییر می‌دهیم.

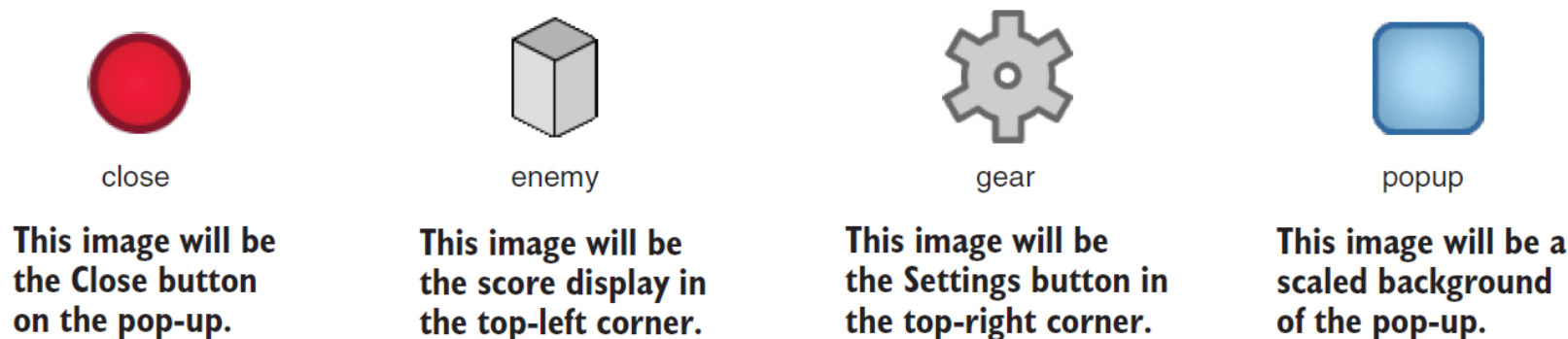
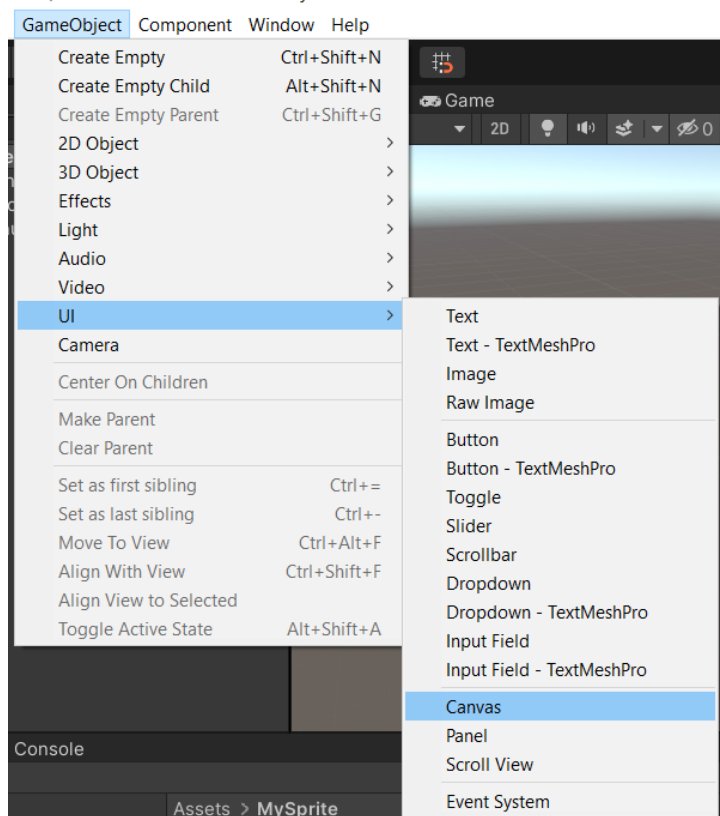


Figure 7.3 Images that are needed for this chapter's project

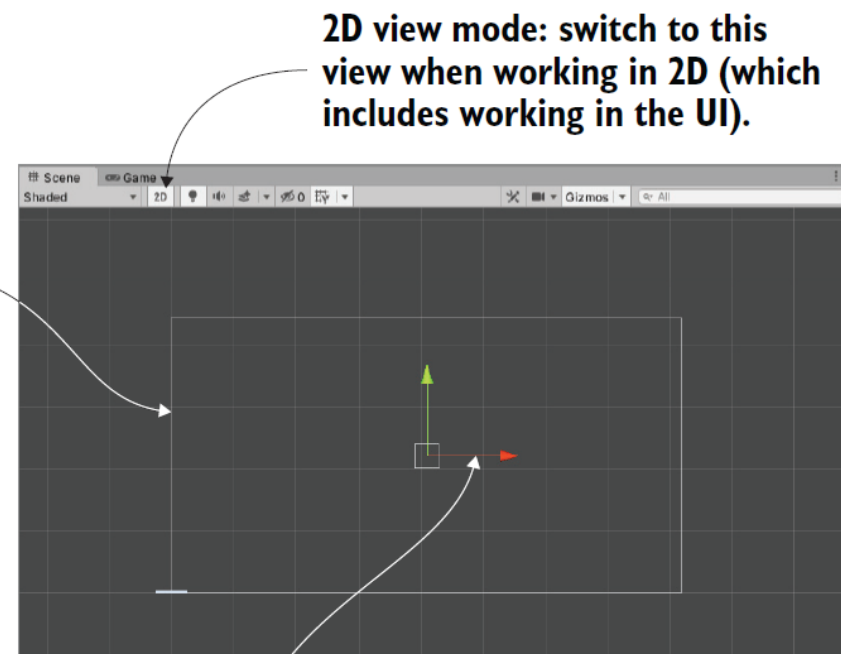
- یکی از اساسی ترین جنبه های نحوه عملکرد سیستم UI این است که همه تصاویر باید به یک شی بوم (canvas) متصل شوند.
- Canvas نوع خاصی از شی است که Unity به عنوان رابط کاربری برای یک بازی ارائه می کند.



Canvas object displayed in the Scene view

It's scaled very large because 1 unit in the scene = 1 pixel on the UI.

The borders of the canvas scale to match the game's screen.

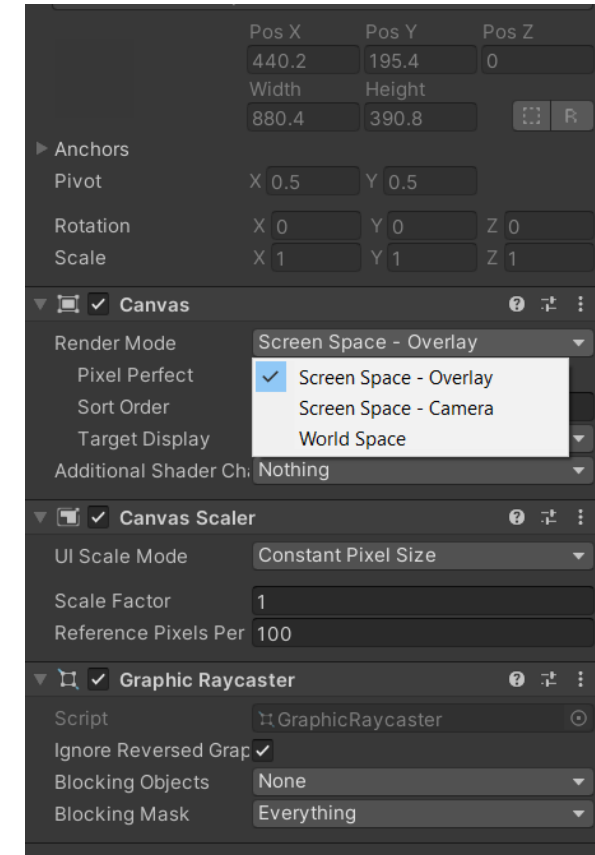


If you see the colored arrows of the manipulator, the Rect tool is not on. That tool button is in the top-left corner of Unity; you will see blue dots on every corner of a 2D object.

Figure 7.4 A blank canvas object in the Scene view

- بوم دارای تنظیماتی است که می توانید آنها را تنظیم کنید. اولین مورد گزینه **Render Mode** است. این را در تنظیمات پیش فرض بگذارید (فضای صفحه-روکش)، اما باید بدانید که سه تنظیمات ممکن به چه معنا هستند:

- **Screen Space—Overlay** رابط کاربری را به صورت گرافیک دوبعدی در بالای نمای دوربین نمایش می دهد. (این تنظیمات پیش فرض است.)
- **Screen Space—Camera** همچنین رابط کاربری را در بالای نمای دوربین نمایش می دهد، اما عناصر رابط کاربری می توانند برای جلوه های پرسپکتیو بچرخند.
- **World Space** شی بوم را در صحنه قرار می دهد، به گونه ای که گویی رابط کاربری بخشی از صحنه سه بعدی است.



- یک متن، یک دکمه و یک تصویر برای UI اضافه کنید.

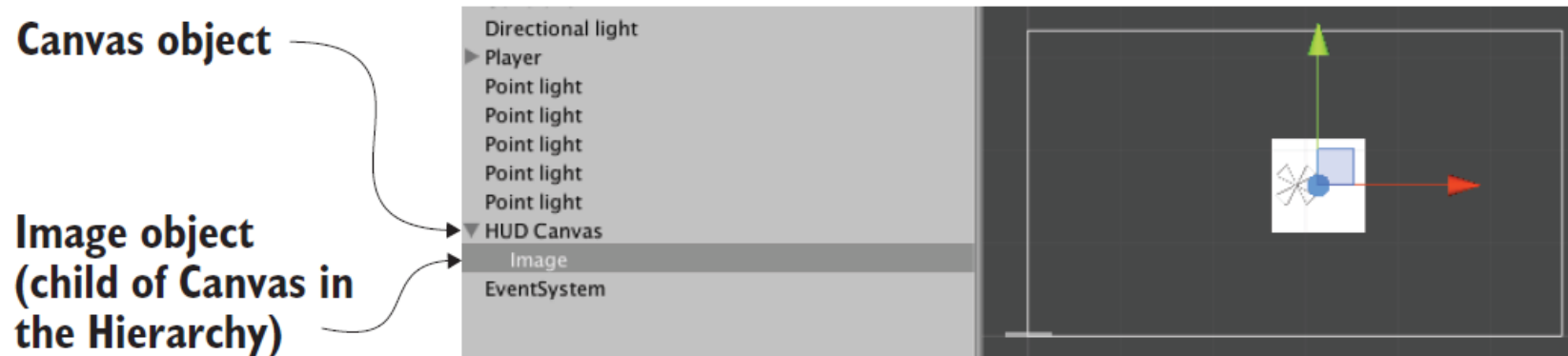
GameObject > UI > Image

Text – TextMeshPro

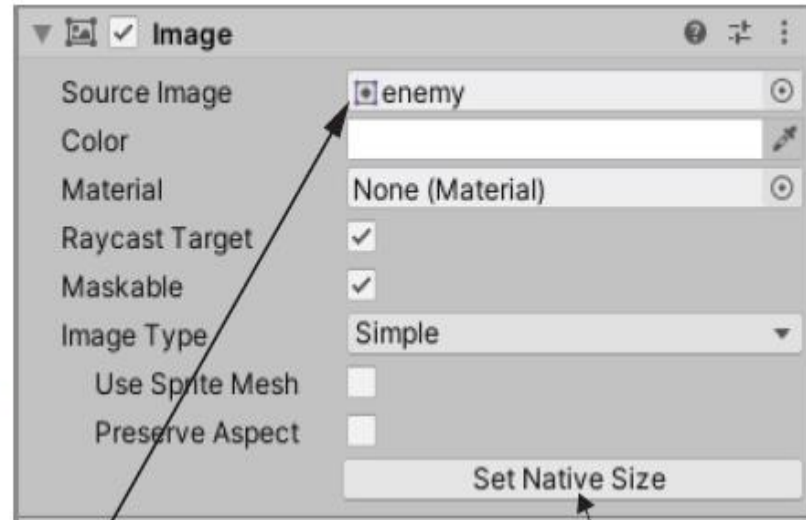
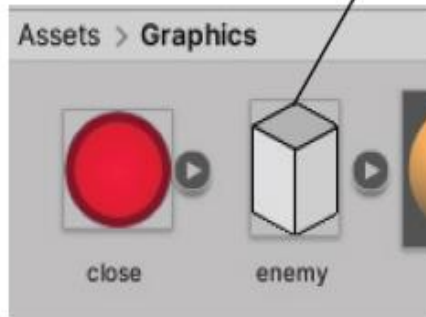
Button - TextMeshPro

NOTE Just as in chapter 5, you need to have the TextMeshPro package installed, so go to Window > Package Manager if no TextMeshPro versions are displayed in the menu of UI objects. The TMP Importer window will automatically appear when you create a TextMeshPro object for the first time. Click the Import TMP Essentials button.

برای نمایش صحیح، عناصر رابط کاربری باید آنها فرزندان شی بوم باشند. یونیتی این کار را به طور خودکار انجام می دهد. شیء دکمه بطور پیش فرض دارای یک شیء متنی است، اما دکمه مورد استفاده در این پروژه به برچسب متنی نیاز ندارد، بنابراین شیء متن پیش فرض را حذف کنید.

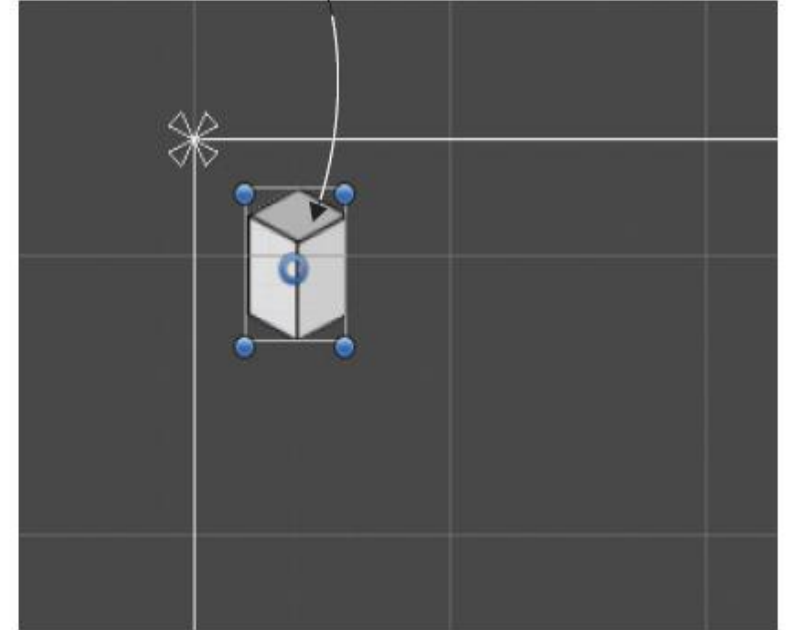


1. Drag the sprite from Project view up to Source Image setting ...

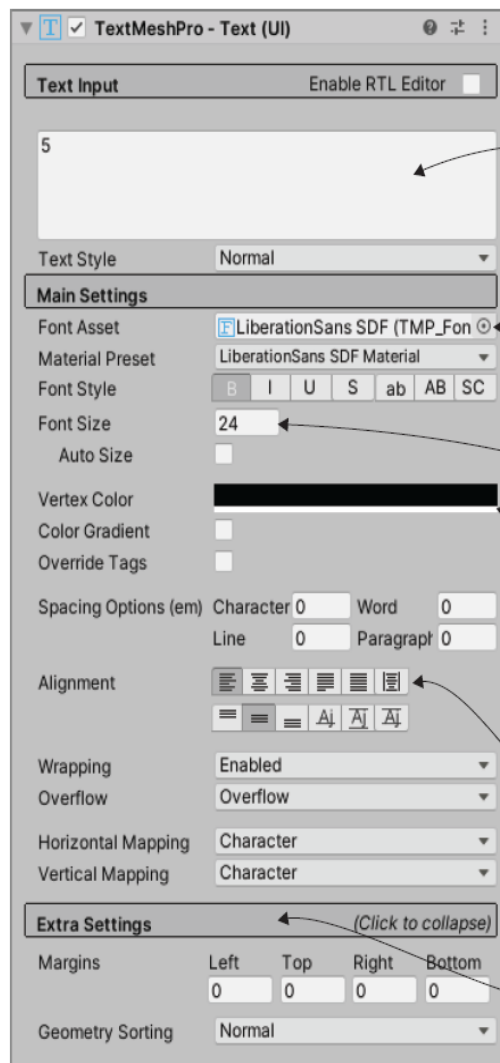


3. Click Set Native Size to resize the image correctly.

2. ... and the image will appear on the UI element.



ابتدا یک عدد را در کادر بزرگ **Text Input** تایپ کنید. این متن بعداً بازنویسی می شود، اما مفید است زیرا مانند نمایش امتیاز در ویرایشگر به نظر می رسد. اندازه متن اشتباه است، بنابراین اندازه قلم را به 24 تغییر دهید. همچنین روی اولین دکمه **Font Style** برای **Bold** کلیک کنید و سپس **Vertex Color** را به سیاه تغییر دهید. همچنین می خواهید این برچسب را روی تراز افقی چپ و تراز عمودی وسط تنظیم کنید. در حال حاضر، تنظیمات باقی مانده را می توان در مقادیر پیش فرض خود باقی گذاشت.



The UI object displays text typed here.

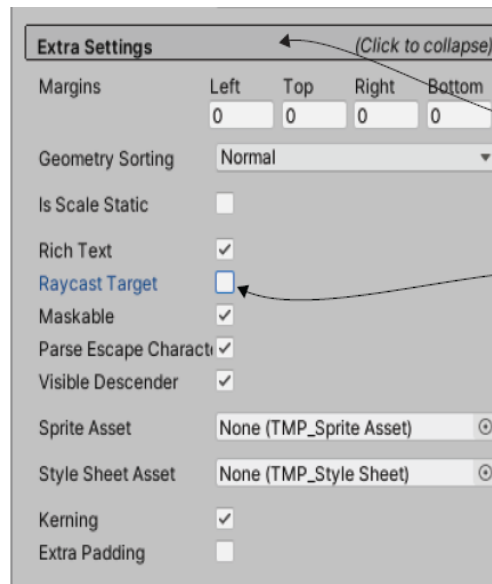
Create a TextMeshPro font asset by using the Font Asset Creator window; set the font here.

Make the text bigger or smaller, as well as Bold or Italic.

Set the color of the text.

These buttons adjust the horizontal and vertical alignment of the text.

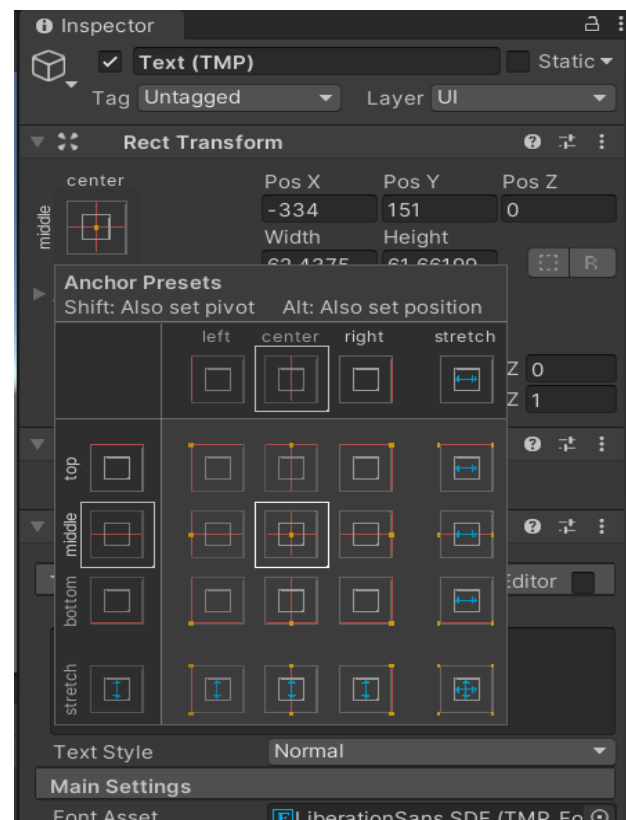
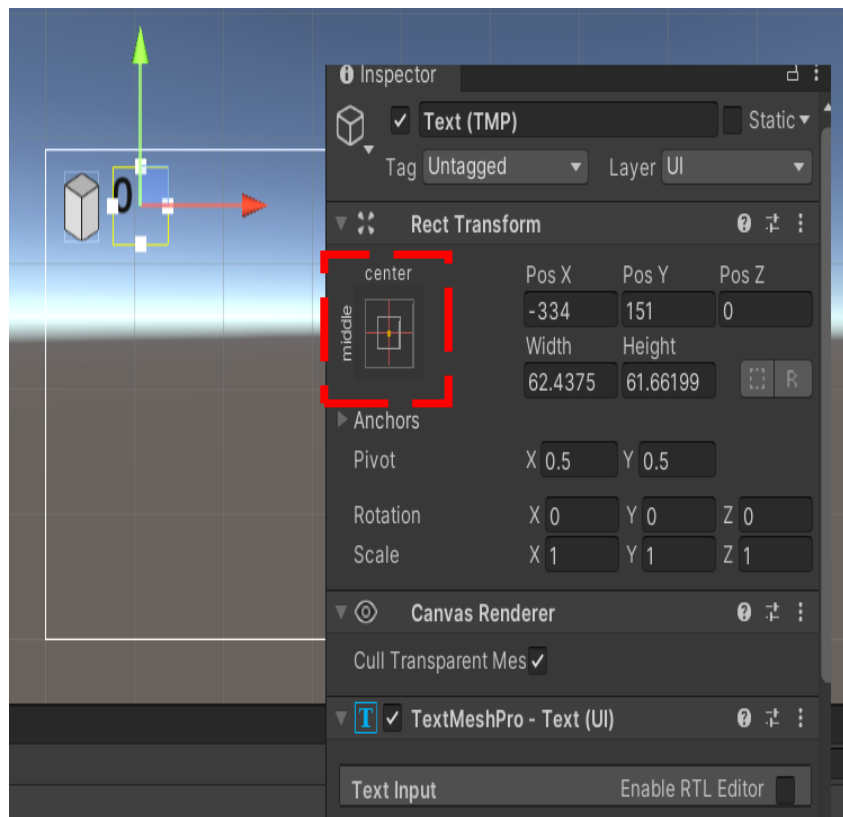
Click this bar to show more settings.



Click this bar to show more settings.

Turn off Raycast Target for text objects that shouldn't be clickable (which is most of them).

- برای هر جسم در این حالت یک نقطه تنظیم و یا لنگر (anchor) تعریف می شود که مکان جسم منو با توجه به آن تنظیم می شود و در صورت تغییر اندازه صفحه نمایش با توجه آن نقطه مکان جسم تنظیم می شود.



هدف لنگر این است که جسم را نسبت به نقطه لنگر در جای خود نگه دارد، در حالی که لنگر نسبت به بوم حرکت می کند. لنگر به عنوان چیزی شبیه "مرکز صفحه" تعریف می شود، و سپس در حالی که اندازه صفحه تغییر می کند، لنگر در مرکز باقی می ماند. به طور مشابه، تنظیم لنگر در سمت راست صفحه، شی را در سمت راست نگه می دارد، حتی اگر اندازه صفحه نمایش تغییر کند (به عنوان مثال، اگر بازی در مانیتورهای مختلف اجرا شود).

Drag the side of the Game view to change the size of the screen.

The canvas scales with it, and the images stay positioned at their anchors in the corners.

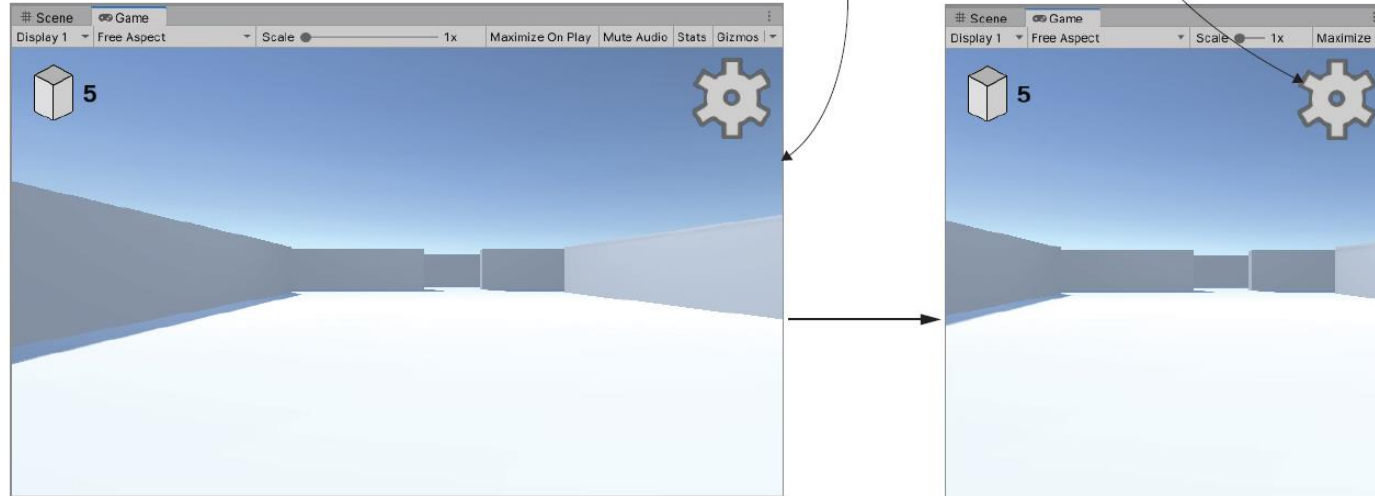


Figure 7.11 Anchors stay in place while the screen changes size.

- در بازی FPS ما موس را غیرقابل مشاهده کرده بودیم که بهتر است آن را با حذف کد مربوطه به حالت اول برگردانیم.
- در هنگام کار با منو GUI بهتر است که بازی اجرا نشود
- می توانید از `EventSystem.current.IsPointerOverGameObject()` استفاده کنید که در هنگام انجام بازی مقدار آن درست است

```
using UnityEngine.EventSystems;  ← Include UI system code frameworks.
...
void Update() {
    if (Input.GetMouseButtonDown(0) &&
        !EventSystem.current.IsPointerOverGameObject()) { ← Italicized code was already in script; shown for reference.
        Vector3 point = new Vector3(
            camera.pixelWidth/2, camera.pixelHeight/2, 0); ← Check that GUI isn't being used.
        ...
    }
```

- گاهی اوقات کنترل های تعامل پیش فرض رابط کاربری نیز با بازی تداخل پیدا می کنند.
- شیء `EventSystem` را که به طور خودکار همراه با بوم ساخته می شود را در نظر بگیرید. آن شیء کنترل های تعامل رابط کاربری را کنترل می کند و به طور پیش فرض از کلیدهای جهت دار برای تعامل با رابط کاربری گرافیکی استفاده می کند. ممکن است لازم باشد برای جلوگیری از تعامل تصادفی با رابط کاربری گرافیکی، کلیدهای جهت نما را خاموش کنید: برای انجام این کار، کادر بررسی `Send Navigation Event` را در تنظیمات `EventSystem` از حالت انتخاب خارج کنید.

- برای ایجاد تعامل با UI باید گام های زیر را بردارید:

1. Create a UI object in the scene (the button created in the previous section).
2. Write a script to call when the UI is operated.
3. Attach that script to an object in the scene.
4. Link UI elements (such as buttons) to the object with that script.

- یک شیء تهی ایجاد کرده و نام آن را controller بگذارید

- یک فایل C# جدید ایجاد کرده و کد زیر را در آن قرار داده و این کامپوننت جدید را به شیء controller اضافه کنید.

- شیء text را که قبلا ایجاد کرده اید در قسمت مربوطه این کامپوننت قرار داده و بازی را اجرا کنید تا زمان اجرا را به شما نشان دهد

```
[SerializeField] TMP_Text scoreLabel;
```

```
void Update()
```

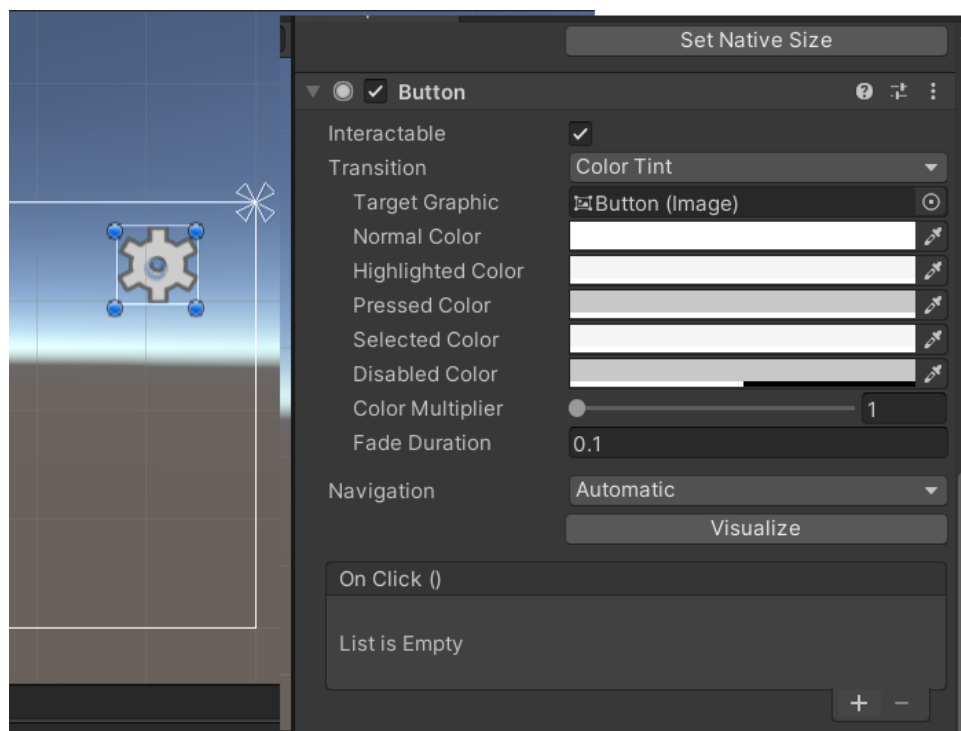
```
{  
    scoreLabel.text = Time.realtimeSinceStartup.ToString();  
}
```

```
public void OnOpenSettings()
```

```
{  
    Debug.Log("open settings");  
}
```

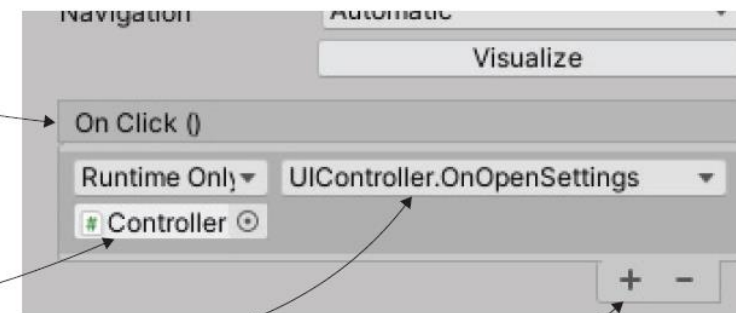
زمانی که یک فیلد بصورت [SerializeField] تعریف می شود مانند یک متغیر public در ویرایشگر یونیتی در دسترس است اما یک تفاوت با متغیر public دارد و آن این است که توسط سایر کلاس ها و کامپوننت ها در دسترس نیست

- اضافه کردن یک رخداد **OnClick** به دکمه منو
- کامپوننت **Button** را اضافه کرده و از قسمت خصوصیات مانند تصویر یک هندل **OnClick** ایجاد کنید.
- مطابق تصویر شیء تهی کنترل را که در مرحله قبل ساختید به آن اضافه کرده
- همچنین تابع **OnOpenSettings** را که در کد قبلی تعریف کردیم به آن اضافه خواهیم کرد.



On Click event panel near the bottom of the settings

Drag an object in the scene to the object slot; then choose a function in the menu.

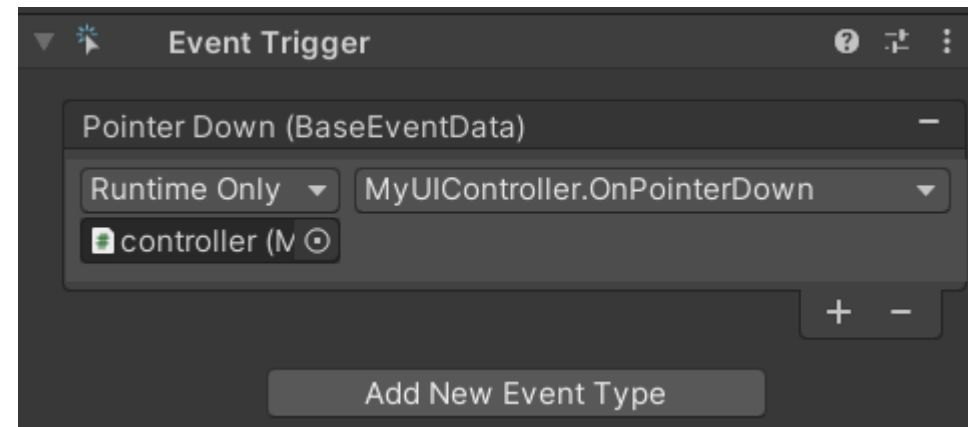


Press the + button to add an entry in the panel.

Figure 7.12 The On Click panel toward the bottom of the button settings

- رخداد **OnClick** تنها در هنگام فشار دادن و رها کردن موس اتفاق می افتد ولی ما می توانیم از سایر رخدادها نیز استفاده کنیم
- برای استفاده از سایر رخداد شما می توانید یک کامپوننت **EventTrigger** را به شیء **GUI** اضافه کرده و یک تابع هندل کننده جدید را در آن فراخوانی کنید. برای مثال تابع `OnPointerDown()` را به همان کامپوننت قبلی اضافه کرده و تنظیمات را مطابق تصویر انجام دهید.

```
public void OnPointerDown()
{
    Debug.Log("pointer down");
}
```



Creating a pop-up window

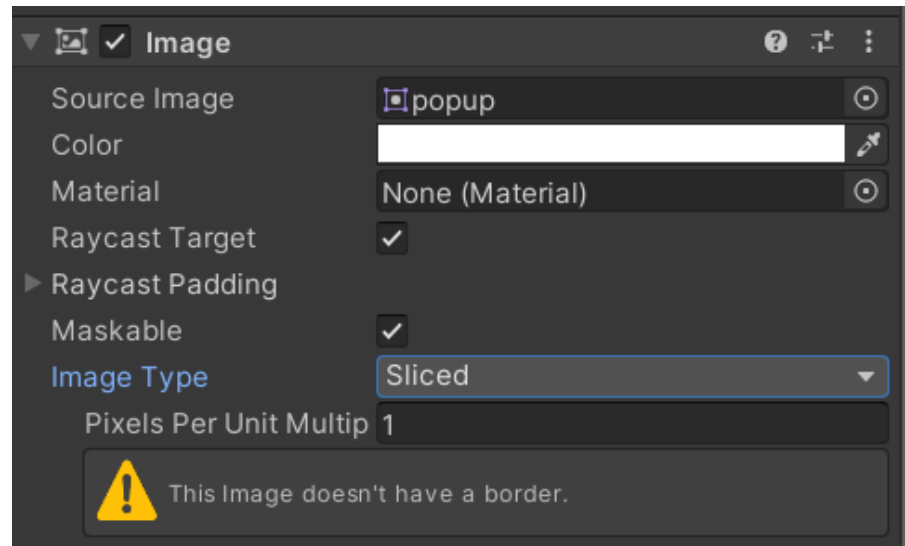
- می خواهیم با کلیک کردن بر روی دکمه چرخ دنده پنجره تنظیمات باز و با کلیک دوباره پنجره مورد نظر بسته شود.

- ابتدا یک شیء image جدید ایجاد کنید GameObject > UI > Image

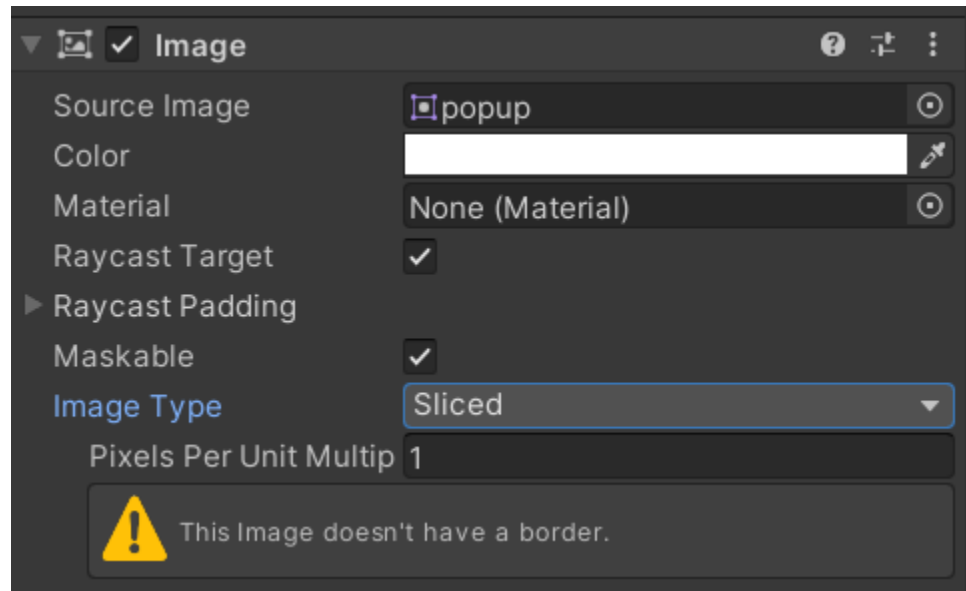
- تصویر Popup را به آن اضافه کنید.

- Set Native Size button to resize the object to the size of the image

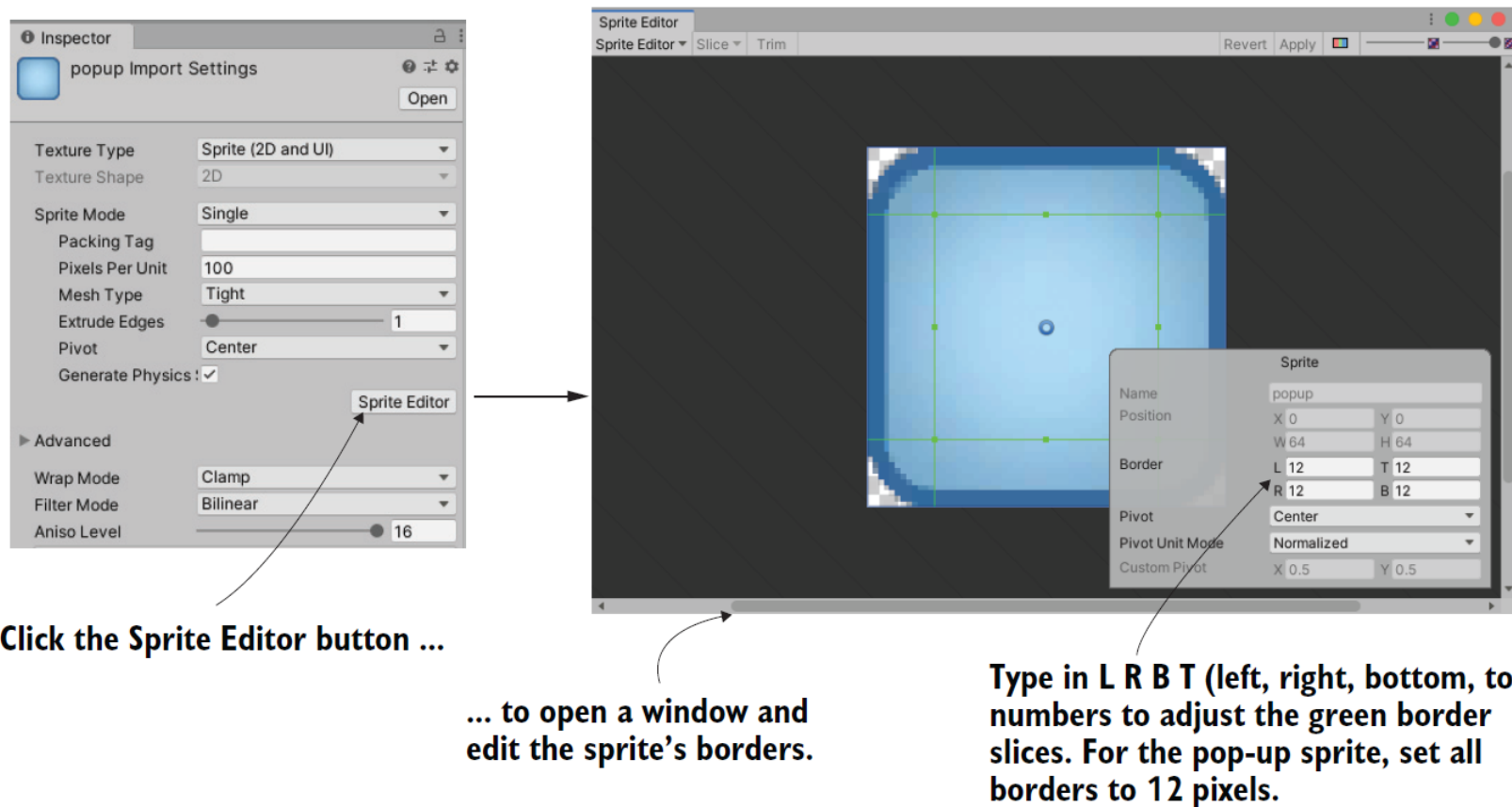
- نوع تصویر را Slice قرار دهید.



- یک تصویر برش خورده به ۹ بخش تقسیم می شود که مقیاس آنها متفاوت از یکدیگر است. با کوچک کردن لبه های تصویر جدا از وسط، اطمینان حاصل می کنید که تصویر می تواند به هر اندازه ای که می خواهید مقیاس شود و لبه های وضوح خود را حفظ می کند.



این خطا به این دلیل رخ می دهد که sprite هنوز 9 بخش مرزی را تعریف نکرده است. برای تنظیم آن، ابتدا sprite را در نمای پروژه انتخاب کنید. در Inspector، باید دکمه Sprite Editor را ببینید. روی آن دکمه کلیک کنید و پنجره Sprite Editor ظاهر می شود.



Click the Sprite Editor button ...

... to open a window and edit the sprite's borders.

Type in L R B T (left, right, bottom, top) numbers to adjust the green border slices. For the pop-up sprite, set all borders to 12 pixels.

Figure 7.14 Sprite Editor button in the Inspector and a pop-up window

در ویرایشگر Sprite می توانید خطوط سبز رنگی را مشاهده کنید که نشان می دهد تصویر چگونه برش داده می شود. در ابتدا، تصویر هیچ حاشیه ای نخواهد داشت (همه تنظیمات Border 0 هستند). (عرض حاشیه را به 12 برای هر چهار طرف افزایش دهید، که منجر به حاشیه نشان داده شده در شکل بالا می شود. از آنجایی که هر چهار طرف (چپ، راست، پایین و بالا) دارای حاشیه با عرض 12 پیکسل هستند، خطوط مرزی به 9 بخش متقاطع می شوند. پنجره ویرایشگر را ببندید و تغییرات را اعمال کنید.

- For this pop-up, enter a width of 250 and a height of 200
- center it on position 0, 0, 0

The way that UI images stack on top of each other is determined by their order in the Hierarchy view. In the Hierarchy list, drag the pop-up object above other UI objects (always staying attached to the canvas, of course). Now move the pop-up around within the Scene view; you can see how images overlap the pop-up window. Finally, drag the pop-up to the bottom of the canvas hierarchy so that it will display on top of everything else.

روشی که تصاویر UI روی هم چیده می شوند با ترتیب آنها در نمای سلسله مراتبی تعیین می شود. در فهرست سلسله مراتب، شی پاپ آپ را روی سایر اشیاء رابط کاربری بکشید (البته همیشه به بوم متصل می ماند). اکنون پاپ آپ را در نمای صحنه حرکت دهید. می توانید مشاهده کنید که چگونه تصاویر با پنجره پاپ آپ همپوشانی دارند. در نهایت، پاپ آپ را به پایین سلسله مراتب بوم بکشید.

- یک کامپوننت با نام `SettingsPopup` را با کد زیر ایجاد کنید.
- کامپوننت را به شیء `UI` که تصویر `pop-up` را نشان می دهد اضافه کنید (تصاویر آبی رنگی که در اسلاید قبل آن را تنظیم کردید)

```
public void Open()
{
    gameObject.SetActive(true);
}
public void Close()
{
    gameObject.SetActive(false);
}
```

- کدهای زیر را به `UIController` اضافه کنید.

```
[SerializeField] MySettingsPopup settingsPopup;
void Start()
{
    settingsPopup.Close();
}
public void OnOpenSettings()
{
    settingsPopup.Open();
}
```

• یک دکمه برای بستن pop-up ایجاد خواهیم کرد

- GameObject> UI> Button – TextMeshPro
- position the new button in the top-right corner of the pop-up
- drag the close sprite to this UI element's Source Image property
- click Set Native Size to correctly resize the image
- Unlike with the previous button , select the text object and type `Close` in the text field
- Reduce Font Size to 14, and set Vertex Color to white.
- Drag this button onto the pop-up object (Image) so that it will be a child of the pop-up window

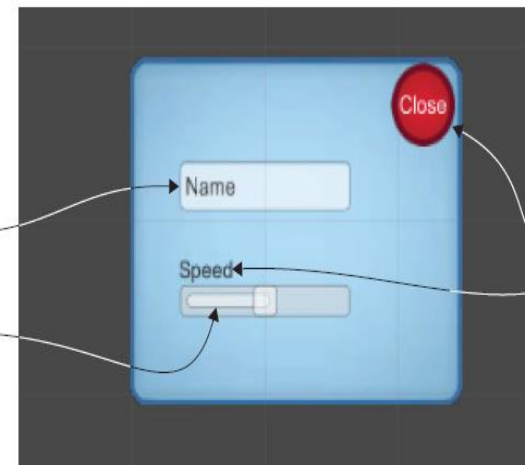
- On the button's On Click panel
- Drag the pop-up window into the object slot
- Choose `SettingsPopup > Close()` from the function list.

- GameObject > UI > InputField – TextMeshPro
- GameObject > UI > Slider
- GameObject > UI > Text - TextMeshPro
- Make all three objects children of the pop-up
- Set the text object to *Speed* and color it black
- Set Max value of spider to 2

Input controls
on the pop-up:

a text InputField

a numerical Slider



The Close button is
in the top corner,
while a text label
is placed just
over the slider.

Figure 7.16 Input controls added to the pop-up window

بهتر است شیء text در روی نوار لغزنده (پایین تر) در سیستم سلسله مراتبی قرار بگیرد

چگونه پیام هایی را بین بازی سه بعدی و GUI جابه جا کنیم

- برای آن که بتوانیم پیام هایی بین بازی و UI جابه جا کنیم از کدی با نام Messenger استفاده می کنیم. این کد را از آدرس زیر به پروژه خود اضافه کنید.
- <https://github.com/jhocking/from-unity-wiki/blob/main/Messenger.cs>
- روند کلی به این صورت خواهد بود که اشیائی که باید از رخدادی مطلع شوند را در یک لیست ثبت می کنیم و زمانی که اتفاق رخ می دهد پیامی را به همه آنها ارسال می کنیم
- شمارش تعداد برخوردها
- شیء متنی گوشه سمت راست صفحه تعداد برخوردها را نشان می دهد برای این کار `UIController` زمانی که فعال می شود خود را برای رخداد ثبت نام می کند و در زمان غیر فعال شدن خود را حذف میکند.
- زمانی که این شیء پیامی را دریافت میکند تابع `OnEnemyHit` را فراخوانی کرده و متن را به روز رسانی میکند.

- ما رخدادهای مورد نظر خود را با استفاده از کلاس `GameEvent` بصورت زیر ثبت می کنیم.

```
public static class GameEvent {  
    public const string ENEMY_HIT = "ENEMY_HIT";  
    public const string SPEED_CHANGED = "SPEED_CHANGED";  
}
```

Adding event listeners to UIController

```
private int score;
void OnEnable() {
    Messenger.AddListener(GameEvent.ENEMY_HIT, OnEnemyHit);
}
void OnDisable() {
    Messenger.RemoveListener(GameEvent.ENEMY_HIT, OnEnemyHit);
}
void Start() {
    score = 0;
    scoreLabel.text = score.ToString();
    settingsPopup.Close();
}
private void OnEnemyHit() {
    score += 1;
    scoreLabel.text = score.ToString();
}
```

- حال باید هنگامی که برخوردی اتفاق می افتد آن را به همه اعلام کنیم. برای این کار باید کد RayShooter را مطابق زیر باز نویسی کرده تا خبر اتفاق به همه کسانی که منتظر شنیدن آن هستند ارسال شود.

```
if (target != null) {  
    target.ReactToHit();  
    Messenger.Broadcast(GameEvent.ENEMY_HIT);  
} else
```

- اضافه کردن رخداد `OnSpeedValue` به کنترل `Spider`
- اضافه کردن `lisener` به بازیکن تا به این وسیله خودش را ثبت کند
- اضافه کردن `Broadcast` به رخداد `OnSpeedValue` تا پیام لازم را به همه ارسال کند.